

A Numerical Analysis of DPM-Solver: Convergence, Stability and Cost in Fast Diffusion Sampling

CSE 402 Final Project Report · Section A · Group 03

Mehemud Azad
Student ID 2105014
Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh

Sayjad Rahman
Student ID 2105021
Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh

Gourove Roy
Student ID 2105017
Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh

Khalid Hasan Tuhin
Student ID 2105002
Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh

Niloy Das Robin
Student ID 2105019
Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh

Code, notebooks and results: <https://github.com/MehemudAzad/NUMERICAL-PROJECT>

Abstract

Diffusion models generate an image by integrating the probability-flow ODE from noise to data, and each step costs one evaluation of a large neural network (one NFE). DPM-Solver produces good samples in 10 to 20 NFE by solving the linear part of this ODE exactly; its authors evaluated it with FID, an image-quality score. We evaluate the same solvers as numerical methods, measuring convergence order, stability near the data end of the trajectory, and error per NFE. Three testbeds supply references of decreasing exactness: an anisotropic Gaussian with a closed-form trajectory, a point mixture integrated with DOP853, and the paper's own CIFAR-10 DDPM network compared against a fine-grid run of itself. Three solver groups separate the paper's two ideas. With an exact score and small steps, every solver reaches its textbook order to within 0.094. At practical budgets (10 to 120 NFE, down to $t = 10^{-3}$) no solver is in that asymptotic regime: even with an exact score the order misses theory by up to 1.91, against 2.79 on the network. Explicit steppers in t become unstable on the curved testbed but not on the linear one, and the λ reparameterisation removes the instability. At matched NFE, third-order DPM-Solver overtakes first order at 14 NFE on the paper's checkpoint, reproducing an unexplained anomaly in the paper's Table 6. At 10 NFE, ranking samplers by FID and by distance to the converged image gives exactly opposite orders.

CCS Concepts

• **Mathematics of computing** → Ordinary differential equations; *Mathematical software*; • **Computing methodologies** → *Neural networks*.

Keywords

diffusion models, probability-flow ODE, DPM-Solver, Runge-Kutta methods, convergence order, numerical stability, exponential integrators

1 Introduction and base paper review

1.1 Background

A diffusion model generates an image by reversing a noising process [9, 20]. In the forward process a real image $x(0)$ is mixed with Gaussian noise, $x(t) = \alpha(t)x(0) + \sigma(t)\varepsilon$, where the signal weight α falls and the noise weight σ rises as t runs from 0 to 1. By $t = 1$ nothing of the image is left (Figure 1a). This direction is a formula and needs no network. A neural network $\varepsilon(x, t)$ is trained to predict the noise contained in $x(t)$, and it is the only learned part of the model.

To generate an image, one starts from pure noise at $t = 1$ and integrates a deterministic ODE, the probability-flow ODE, back to a small time t_{end} , where the state is an image [20] (Figure 1b). At every step the network's prediction also gives an estimate of the finished image, $\hat{x}_0 = (x - \sigma\varepsilon)/\alpha$; in the figure the layout of the picture is already fixed by $t \approx 0.4$, and the remaining steps add detail. Each evaluation of the ODE's right-hand side is one forward pass of the network, and that pass dominates the cost. The number of function evaluations (NFE) is therefore the unit of cost, and the practical question is how few evaluations a solver can take before the result degrades. Early samplers used 100 to 1000 steps [9, 19].

1.2 The base paper

Lu et al. [15] observe that the probability-flow ODE is semi-linear. Its right-hand side is a linear term in the state x , with a known coefficient, plus a term that contains the network. Classical explicit solvers such as Runge-Kutta methods [1, 6] approximate both terms. DPM-Solver integrates the linear term exactly with the variation-of-constants formula and approximates only the network term, by a Taylor expansion in the half log signal-to-noise ratio λ . Stepping uniformly in λ also places more steps near the data end, where the ODE stiffens.

The paper derives solvers of order 1, 2 and 3 (DPM-Solver-1 is algebraically the same update as DDIM [19]) and proves that

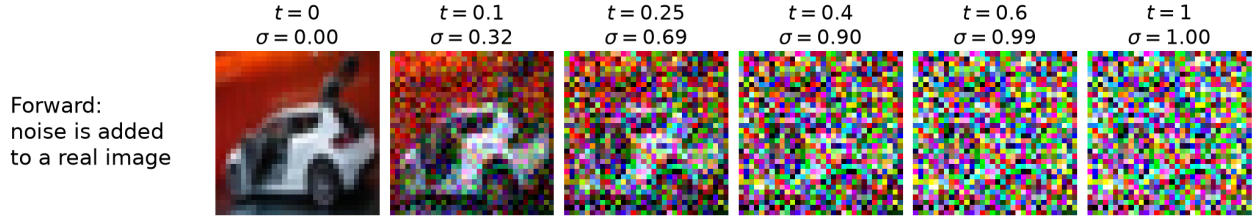
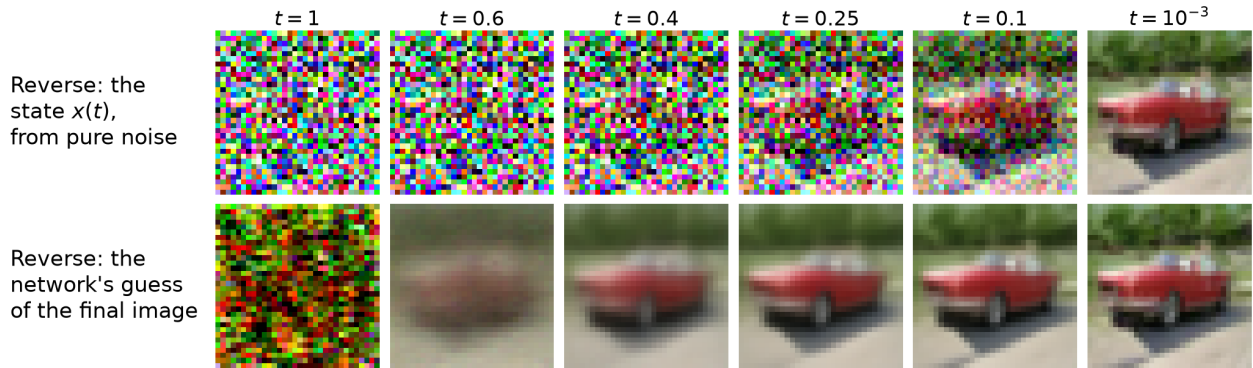
(a) Forward process: a formula, no network needed**(b) Reverse process: DPM-Solver-3 solves the probability-flow ODE (45 network calls)**

Figure 1: From noise to an image, on the CIFAR-10 checkpoint we study. (a) The forward process mixes a real CIFAR-10 test image with Gaussian noise; at $t = 1$ the image is gone. (b) Generation runs the other way: DPM-Solver-3 integrates the probability-flow ODE from pure noise at $t = 1$ to $t = 10^{-3}$ with the google/ddpm-cifar10-32 network, in five segments that stop at the times shown (45 network calls). Top row: the state $x(t)$ that the solver carries. Bottom row: the network's estimate of the finished image at that moment, $\hat{x}_0 = (x - \sigma\epsilon)/\alpha$. Panel (b) is a newly generated image, not the one in (a).

DPM-Solver- k converges with order k , under an assumption (its Assumption B.1) that the λ -derivatives of the network output exist and are continuous up to order $k + 1$. It evaluates the solvers with FID [8] and reports good samples in about 10 to 20 NFE. Two further statements in the paper motivate parts of our study. Its Section 4.2 says, citing the exponential-integrator literature [10], that classical explicit solvers may become unstable at large step sizes; it does not measure a stability limit. Its Table 6 reports, without explanation, that on the CIFAR-10 DDPM checkpoint at about 10 NFE the third-order solver scores FID 24.37 against 16.69 for the first-order one.

1.3 Objective of this work

FID measures whether a set of samples resembles real data. It does not measure how accurately a solver followed the ODE, and a sample can look plausible after the solver has left the true trajectory. Our objective is to evaluate DPM-Solver, and the classical solvers it is compared against, by the standards of numerical analysis. We ask whether each solver attains its theoretical order when measured against a known solution, how large a step each solver can take near the stiff end before it diverges, and, once cost is counted in NFE, when a higher-order solver beats a lower-order one. We then put the numerical verdict next to the image-quality verdict on the same checkpoint.

The contributions are:

- a three-arm design that separates DPM-Solver's two ideas, run on three testbeds whose references range from a closed form to a fine-grid run of a real network;
- order measurements under two protocols, which show that most of the order shortfall seen on the real network already appears with an exact score at practical budgets;
- a stability study that finds no instability on a linear problem at any condition number, explains why, and finds a real stability limit on a curved problem;
- a crossover analysis that reproduces the paper's Table 6 anomaly at 14 NFE on the paper's own checkpoint;
- a comparison of trajectory error and FID on the same samplers, which rank them in opposite orders at 10 NFE.

All code, notebooks and results are public at <https://github.com/MehemudAzad/NUMERICAL-PROJECT> [2].

2 Methodology

2.1 Noise schedule and the probability-flow ODE

We use the variance-preserving linear schedule of the base paper, with $\beta_0 = 0.1$, $\beta_1 = 20$ and t running from $T = 1$ to t_{end} :

$$\begin{aligned} \log \alpha(t) &= -\frac{\beta_1 - \beta_0}{4} t^2 - \frac{\beta_0}{2} t, & \sigma(t) &= \sqrt{1 - \alpha(t)^2}, \\ f(t) &= \frac{d \log \alpha}{dt} = -\frac{1}{2} \beta(t), & g^2(t) &= \beta(t) = \beta_0 + t(\beta_1 - \beta_0). \end{aligned} \quad (1)$$

The half log-SNR $\lambda(t) = \log \alpha(t) - \log \sigma(t)$ decreases strictly in t , from $\lambda(1) \approx -5.03$ towards $+\infty$ as $t \rightarrow 0$, and its inverse has the cancellation-free form

$$t(\lambda) = \frac{2u}{\sqrt{\beta_0^2 + 2(\beta_1 - \beta_0)u + \beta_0}}, \quad u = \log(e^{-2\lambda} + 1). \quad (3)$$

2.2 Three solver arms

Write $\varepsilon(x, t)$ for the noise-prediction model: a network in general, an exact formula on our analytic testbeds. The same trajectory can be discretised in three ways, and these define our three arms:

$$(A) \quad \frac{dx}{dt} = f(t)x + \frac{g^2(t)}{2\sigma(t)} \varepsilon(x, t), \quad (4)$$

$$(B) \quad \frac{dx}{d\lambda} = \hat{\sigma}(\lambda)^2 x - \hat{\sigma}(\lambda) \varepsilon(x, t(\lambda)), \quad (5)$$

$$(C) \quad x_i = \frac{\alpha_i}{\alpha_{i-1}} x_{i-1} - \sigma_i (e^{h_i} - 1) \varepsilon(x_{i-1}, t_{i-1}), \quad (6)$$

with $h_i = \lambda_i - \lambda_{i-1}$ and $\hat{\sigma}(\lambda) = (1 + e^{2\lambda})^{-1/2}$. Arm A applies classical steppers to Eq. (4) on a grid uniform in t . Arm B applies the same steppers to Eq. (5), the same ODE written in λ , on a grid uniform in λ . Arm C is DPM-Solver, whose first-order update is Eq. (6). Comparing A with B isolates the change of variable, and comparing B with C isolates the exact treatment of the linear term (Figure 2).

The classical steppers are forward Euler, the explicit midpoint rule (RK2), Heun’s third-order method with nodes 1/3 and 2/3 (RK3), the classical fourth-order Runge-Kutta method (RK4), and the two-step Adams-Bashforth method (AB2, started with one midpoint step). The RK2 and RK3 variants are the ones in the base paper’s Appendix E.4, so arms A and B match the paper’s own baselines. For arm C we use the authors’ code [14], unmodified, with one order throughout the run (singlestep_fixed), a grid uniform in λ (logSNR) and no final denoising step. DPM-Solver- k costs k NFE per step. Everywhere, $e^h - 1$ is computed with `expm1`, because the direct form loses relative precision for the small h a convergence study needs.

2.3 Testbeds with known answers

Measuring error requires the correct answer, which does not exist for a real generated image. We therefore use three testbeds (Figure 3).

Tier 1: anisotropic Gaussian. For data drawn from $\mathcal{N}(0, \text{diag}(s))$ the ODE splits into d independent linear scalar ODEs, and the score

and trajectory are both closed form:

$$\begin{aligned} \varepsilon^*(x, t)_j &= \frac{\sigma(t) x_j}{v_j(t)}, & v_j(t) &= \alpha(t)^2 s_j + \sigma(t)^2, \\ x_j(t) &= x_j(T) \sqrt{v_j(t)/v_j(T)}. \end{aligned} \quad (7)$$

The stiffness parameter is the condition number $\kappa = s_{\max}/s_{\min}$. Because the coordinates decouple, the dimension does not change the dynamics, so we sweep κ and keep d small.

Tier 2: point mixture. With data placed on K points μ_k , the score stays closed form but becomes nonlinear in x , because the posterior mean is a softmax over the modes:

$$r_k \propto w_k \exp\left(-\frac{\|x - \alpha\mu_k\|^2}{2\sigma^2}\right), \quad \hat{x}_0 = \sum_k r_k \mu_k, \quad \varepsilon^* = \frac{x - \alpha\hat{x}_0}{\sigma}. \quad (8)$$

We place $K = 8$ modes on a Swiss-roll curve (the testbed called `mog8`). The responsibilities are computed with `logsumexp`, since a direct exponentiation overflows at small σ . No closed-form trajectory exists, so the reference is SciPy’s DOP853 [3, 6, 21].

Tier 3: the real network. `google/ddpm-cifar10-32` [5, 9], the DDPM checkpoint the paper uses for its discrete-time CIFAR-10 [13] experiments ($d = 3072$). The reference is a 201-NFE DPM-Solver-3 run of the same network from the same starting noise x_T , so the measurement is discretisation error and contains no model error.

2.4 The Tier-3 schedule

The checkpoint was trained with 1000 discrete noise levels, and the authors’ code interpolates $\log \alpha$ linearly between them. This discretised schedule differs from the continuous one by up to 4.74×10^{-2} in λ and 5.0% in f , so reusing the continuous coefficients for arms A and B would add a systematic error to every Tier-3 measurement. We read all coefficients from the checkpoint’s own schedule instead. Arm B needs only $t(\lambda)$ and $\hat{\sigma}(\lambda)$, and the latter follows from $\lambda = \log \alpha - \log \sigma$ and $\alpha^2 + \sigma^2 = 1$ for any variance-preserving schedule. Arm A needs f and g^2 . Differentiating $\sigma^2 = 1 - \alpha^2$ gives $\sigma' = -\alpha^2 f / \sigma$, hence

$$\lambda' = f - \frac{\sigma'}{\sigma} = \frac{f}{\sigma^2}, \quad f = \sigma^2 \lambda', \quad g^2 = -2\sigma^2 \lambda' = -2f. \quad (9)$$

We compute λ' once by a central difference and take both coefficients from it, so $f/\lambda' = \sigma^2$ holds to 1.1×10^{-16} . That identity is what makes Eqs. (4) and (5) the same ODE, and a unit test confirms it: integrating both arms at four resolutions, the gap between them falls from 2.5×10^{-4} to 3.9×10^{-7} .

2.5 Measurement procedures

Order. A measured convergence curve is straight only in its middle: at large h the asymptotic theory does not yet apply, and at small h rounding error takes over. We fit $\log(\text{error})$ against $\log h$ over every contiguous window of at least four points and keep the window that is both straight and wide, scored as $R^2 + 0.01 \times (\text{width})$. We report the slope, the window and R^2 . On Tier 3 the reference has its own error floor (below), and the window search can fit through it; there we also report slopes refitted after discarding points within three times the floor.

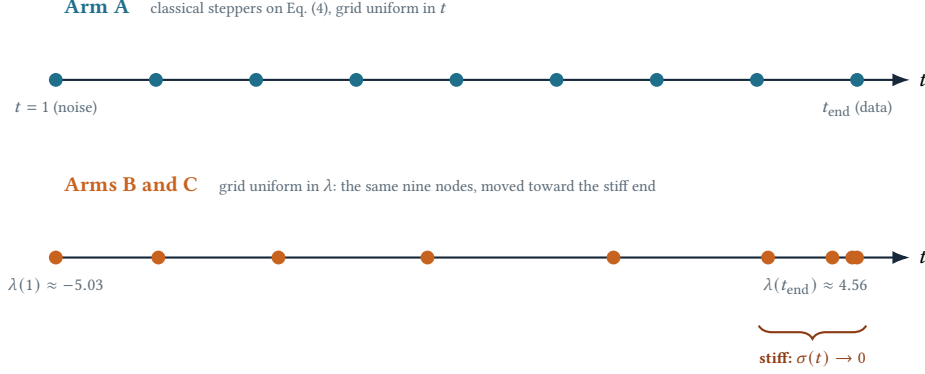


Figure 2: The three arms. All three integrate one trajectory. Arm A steps uniformly in t ; arms B and C step uniformly in λ , which places the nodes (computed here from the schedule) near the stiff end. Arms B and C share a grid but not an update: arm B approximates the whole right-hand side of Eq. (5), while arm C integrates its linear term exactly.

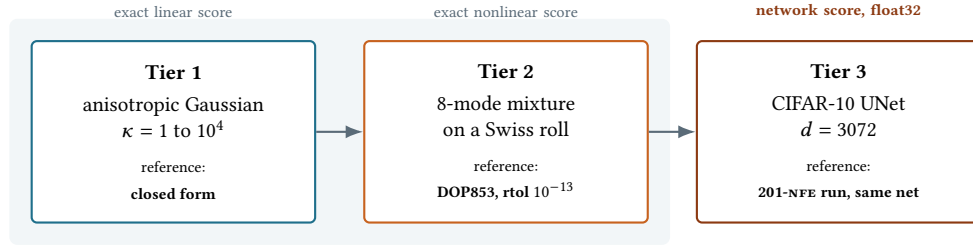


Figure 3: The three testbeds. Moving right trades exactness of the reference for realism of the problem. The shaded pair have analytic scores, so any shortfall from theory there comes from the solver and the budget and cannot come from a model.

Stability. For each stepper we bisect on the number of steps, between 2 and 4096, for the coarsest grid whose output stays finite and below ten times $\|x_T\|$, and report the corresponding largest stable step $h_{\max}$. The bisection is checked against a synthetic linear ODE with a known stability threshold.

Crossover. To find where order 3 starts to beat order 1, we interpolate the error curves e_3 of DPM-Solver-3 and e_1 of DPM-Solver-1 linearly in log-log space and report every sign change of $\log e_3 - \log e_1$, since the curves can cross more than once. We compare the two solvers at matched NFE, which is the comparison in the paper’s Table 6.

Accuracy versus image quality. Trajectory error is the L_2 distance to the converged image, also reported per image in 8-bit gray levels. Image quality is FID [8], computed with clean-fid [17].

3 Implementation and experiments

3.1 Software and hardware

The numerical engine is plain Python (NumPy [7] and SciPy [21]) with no GPU code. It contains the schedule, the testbeds, the steppers and one integrator that counts every call to the right-hand side, so NFE is measured and never inferred from a formula. The same integrator runs all three tiers: it works on float64 NumPy arrays on Tiers 1 and 2 and on float32 PyTorch [18] CUDA tensors on Tier 3, and the probability-flow ODE is defined once, parameterised

by a schedule object. Tier 3 loads the checkpoint with Diffusers [22]. Tables are assembled with pandas [16] and figures drawn with Matplotlib [11]. Tiers 1 and 2 run on a laptop CPU in float64; Tier 3 runs on a Kaggle NVIDIA T4 GPU [12]. The repository has 129 unit tests, and `run_all.sh` regenerates every laptop-side result from a clean clone. Re-running it reproduced the committed sweeps to 1.8×10^{-15} and the fitted slopes to 4.2×10^{-5} .

3.2 Two protocols

Where a convergence study is run, and at what budget, changes what it measures. We use two settings (Table 1). The asymptotic protocol stays away from the data end and starts each sweep past the solver’s pre-asymptotic region, which is the right setting for checking that an implementation has the order promised by the theorem. The practitioner protocol is the one a user runs and the only one affordable on Tier 3. Every comparison across tiers uses the practitioner protocol.

3.3 Parameters and data

Table 2 lists the settings. Every run uses a fixed seed and records a row in one shared results schema.

The benchmark is the base paper itself: its Table 6 (FID of DPM-Solver-1, -2, -3 and DPM-Solver-fast on this checkpoint, $\epsilon = 10^{-3}$) and its Table 4 (DDIM on the same checkpoint). Two samplers that our sweeps had not used are added for the FID comparison because

Table 1: Measurement protocols.

Protocol	Setting	Purpose
asymptotic	$t \in [0.2, 1]; \geq 16$ to 64 NFE, chosen per solver	test the order theorem on Tiers 1 and 2
practitioner	$t \in [10^{-3}, 1]; 9$ to 120 NFE	what users run; Tier 3, and Tiers 1 and 2 re-run for comparison

Table 2: Experimental settings.

Item	Value
Schedule	VP linear, $\beta_0 = 0.1, \beta_1 = 20, T = 1$
Tier 1	$d = 3; \kappa \in \{1, 10, 10^2, 10^3, 10^4\}$, stability stress test to 10^8
Tier 2	mog8: 8 equal-weight modes on a Swiss roll
Tier 2 reference	DOP853, rtol 10^{-13} , atol 10^{-14} ; rerun at 10^{-11} agrees to 6.9×10^{-13}
Tier 3 batch	64 images, seed 0, float32
Tier 3 references	DPM-Solver-3 at 201 NFE; 300 NFE for the floor; $t_{\text{end}} \in \{2, 5, 10\} \times 10^{-3}$ for truncation
Tier 3 budgets	10, 12, 15, 20, 30, 50, 80, 120 NFE
Sample grid	6 samplers $\times$ 5 budgets on 8 noises; 64 images per sampler for per-image error
FID	5,000 samples per setting, clean-fid, legacy_tensorflow, CIFAR-10 training statistics
FID floor	real CIFAR-10 test images: three draws each of 1,000, 2,000 and 5,000; the full 10,000 once

the paper uses them: DPM-Solver-fast, which mixes orders 1 to 3 to spend the budget exactly, and DDIM on its quadratic time grid.

3.4 Validation

The paper shows that DDIM and DPM-Solver-1 are algebraically the same update, so two independent implementations must agree to machine precision. Ours agree to 5.5×10^{-16} , and DPM-Solver-1 measures as order 0.99; nothing downstream was run before this check passed. The Tier-1 closed form satisfies the ODE to about 10^{-11} and matches an independent DOP853 integration to 10^{-12} , and our continuous schedule agrees with the one in the authors' code to 10^{-11} , so all three arms share one schedule.

4 Results and discussion

We ran seven experiments. Table 3 lists what each one asks and what it found; the subsections below give the measurements behind each row.

4.1 Order with an exact score

Under the asymptotic protocol every solver in every arm reaches its theoretical order (Table 4, Figure 4). The largest deviation is 0.094 on Tier 1 and 0.084 on Tier 2, with mean R^2 of 0.99992 and 0.99996, and moving from the linear to the curved testbed shifts no slope by more than 0.059. Both testbeds have analytic scores, so Assumption B.1 holds and the theorem's conclusion holds with it. The result

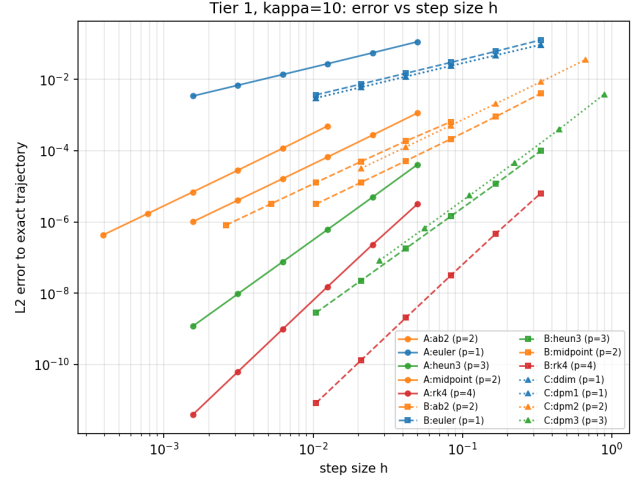


Figure 4: Tier 1 ($\kappa = 10$), asymptotic protocol: error against step size for every solver in every arm. The slope of each line is its measured order (Table 4). DDIM and DPM-Solver-1 lie on top of each other.

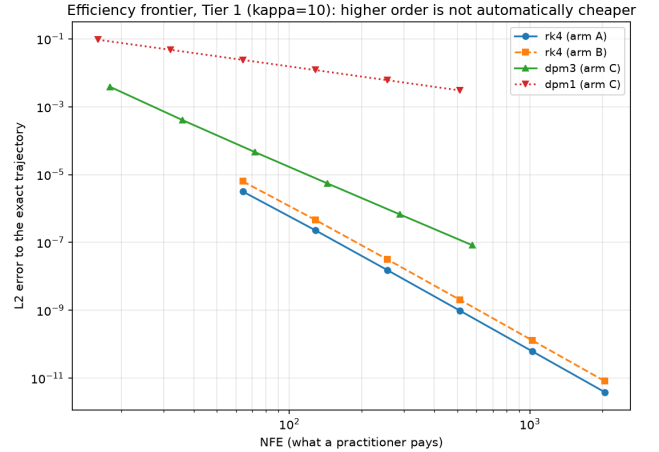


Figure 5: Tier 1 ($\kappa = 10$), asymptotic protocol: error against measured NFE, the cost a user pays. RK4 in either variable stays below DPM-Solver-3 at every shared budget.

also confirms that the published code, driven by an exact oracle, implements the method correctly.

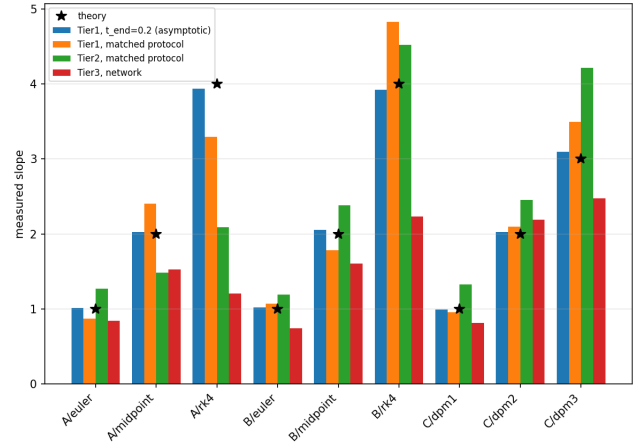
Cost per NFE. Order is defined per step, but a step of RK4 costs four network calls and a step of DPM-Solver- k costs k . Figure 5 puts the Tier-1 errors on a common NFE axis. In the asymptotic regime the higher order pays for its extra calls: RK4 in t is 21 times more accurate than DPM-Solver-3 at 64 NFE and 121 times at 512. The advantage the paper claims for DPM-Solver lies at 10 to 20 NFE, which these sweeps never visit. Tier 3 does, and there DPM-Solver-3 beats RK4 per NFE from 16 to 120 NFE (Figure 10).

Table 3: The seven experiments at a glance: the question each one asks, and the answer the measurements give.

Experiment	Question	Result	Section
1 DDIM $\equiv$ DPM-Solver-1	Is our implementation exact?	The two updates agree to 5.5×10^{-16} ; DPM-Solver-1 measures order 0.99.	3.4
2 Convergence order	Does the error fall at the textbook rate?	With an exact score and small steps, every solver is within 0.094 of theory. At 10 to 120 NFE down to $t = 10^{-3}$ the order misses theory by up to 1.91 with an exact score and 2.79 on the network.	4.1, 4.2
3 Stability near $t \rightarrow 0$	Do large steps diverge near the data?	Linear testbed: never, up to $\kappa = 10^8$. Curved testbed: RK4 in t is limited to $h \leq 0.1998$. Stepping in λ : never.	4.3
4 Crossover at equal cost	When does order 3 beat order 1?	From 5.1, 9.2 and 14.1 NFE on Tiers 1, 2 and 3; the paper’s Table 6 puts it between 10 and 12.	4.6
5 Change of variable (A vs B)	What does stepping in λ buy?	RK4’s error falls $48\times$, $8551\times$ and $10.6\times$ on the three tiers; for Euler and RK2 the effect is mixed.	4.4
6 Exact linear part (B vs C)	When does it help?	On concentrated data (ahead at 83 to 100% of points for $s < 10^{-2}$). For data as spread out as the noise ($s = 1$) classical steppers are exact and DPM-Solver-1 is not.	4.5
7 Image quality vs accuracy	Is the best-looking sampler the most accurate?	No. At 10 NFE FID and distance to the converged image rank the samplers in exactly opposite order (rank correlation -1.00 ; $+0.60$ at 20 NFE). Sample size explains about half of the gap to the paper’s FIDs.	4.8

Table 4: Measured order with an exact score, asymptotic protocol.

Arm	Solver	p	Tier 1	Tier 2	max $ \Delta $
A	Euler	1	1.012	1.012	0.012
A	midpoint	2	2.028	2.031	0.031
A	Heun3	3	3.014	2.999	0.014
A	RK4	4	3.938	3.997	0.062
A	AB2	2	2.033	1.988	0.033
B	Euler	1	1.023	1.023	0.023
B	midpoint	2	2.057	2.055	0.057
B	Heun3	3	3.015	3.024	0.024
B	RK4	4	3.921	3.965	0.079
B	AB2	2	1.935	1.975	0.065
C	DDIM	1	0.994	0.996	0.006
C	DPM-1	1	0.994	0.996	0.006
C	DPM-2	2	2.028	2.018	0.028
C	DPM-3	3	3.094	3.084	0.094

**Figure 6: Measured slopes against theory (stars). Tier 1 under the asymptotic protocol, Tiers 1 and 2 under the practitioner protocol, and Tier 3. The large departures appear with an exact score once the protocol matches Tier 3’s.**

4.2 Order at practical budgets

Table 5 repeats the measurement under the practitioner protocol on all three tiers. With an exact score and nothing else changed except the interval and the budget, the slopes scatter on both sides of theory by up to 1.91: RK4 in t on Tier 2 measures 2.09 and RK4 in λ measures 4.52. At 9 to 120 NFE, integrating down to 10^{-3} , no solver is in the asymptotic regime its order describes, with or without a network. The exact-score Tier 2 already shows 68% of the network’s worst gap (1.91 of 2.79; Figure 6).

The network adds a further downward shift, largest for the high-order steppers: measured against the matched Tier-2 control, RK4 in t falls from 2.09 to 1.21, RK4 in λ from 4.52 to 2.23 and DPM-Solver-3 from 4.21 to 2.47. DPM-Solver-3’s value is affected by the reference floor: discarding the one sweep point that reaches it gives

2.995, close to its theoretical order. Curves far above the floor, such as RK4 in t at twelve times the floor, do not change.

An earlier version of our analysis attributed the Tier-3 shortfall to the network violating Assumption B.1. The matched control does not support that. The assumption only requires the λ -derivatives of ε to exist and be continuous, and this UNet, built from SiLU activations [4], normalisation and sinusoidal time embeddings, is a smooth function of its inputs. A network can make those derivatives large, which moves the start of the asymptotic regime to smaller steps than any practical budget reaches.

Table 5: Measured order under both protocols. Floor-excl. refits Tier 3 after discarding points within three times the reference floor.

Arm	Solver	p	asymptotic		practitioner ($t_{\text{end}} = 10^{-3}$, 9 to 120 NFE)			
			Tier 1	Tier 2	Tier 1	Tier 2	Tier 3	floor-excl.
A	Euler	1	1.012	1.012	0.871	1.269	0.842	0.842
A	midpoint	2	2.028	2.031	2.403	1.485	1.525	1.500
A	RK4	4	3.938	3.997	3.297	2.093	1.210	1.210
B	Euler	1	1.023	1.023	1.073	1.191	0.746	0.746
B	midpoint	2	2.057	2.055	1.786	2.385	1.603	1.603
B	RK4	4	3.921	3.965	4.829	4.523	2.231	1.099 [†]
C	DPM-1	1	0.994	0.996	0.958	1.325	0.813	0.813
C	DPM-2	2	2.028	2.018	2.100	2.455	2.191	2.244
C	DPM-3	3	3.094	3.084	3.492	4.214	2.473	2.995
worst $ \text{slope} - p $			0.094	0.084	0.83	1.91	2.79	

[†]Two of eight points lie on the floor; the six left are mostly pre-asymptotic, so neither Tier-3 value is reliable for B/RK4.

4.3 Stability

Linear testbed. For every κ from 1 to 10^4 , every stepper in both arms is stable on the coarsest grid tried (two steps), and the stress test to $\kappa = 10^8$ and $t_{\text{end}} = 10^{-6}$ changes nothing. The cause is structural. On a uniform- t grid the node nearest the boundary is at $t_{\text{end}} + h$, and the stability product $h J(t_{\text{end}} + h)$, with J the local Jacobian, never exceeds about 0.9, below explicit Euler’s threshold of 2. Near the boundary $\sigma(t)^2 \approx \beta_0 t$, so the growth of the local rate and the shrinking distance to t_{end} almost cancel, and for $s_{\text{min}} \ll \sigma^2$ the rate is capped by σ^2 and does not depend on κ .

Curved testbed. On Tier 2 the Jacobian has no such cancellation (Figure 7). As $\sigma \rightarrow 0$ the posterior weights switch more sharply between modes, and at $t = 10^{-3}$ the spectral norm of the arm-A Jacobian is 545, against 0.54 on Tier 1. RK4 in t now has a real limit, $h_{\text{max}} = 0.1998$: it is stable from 5 steps and diverges at 2 to 4. A stricter divergence rule gives the same limit. RK4 is the only arm-A stepper whose last stage evaluates the right-hand side at the end of the step, which on the final step is t_{end} , where the Jacobian peaks. In arm B every stepper is stable at every grid tried.

The paper’s Section 4.2 statement therefore holds in a narrower form: explicit steppers in t can become unstable near the data, the instability comes from the curvature of the data distribution and does not depend on κ , and the λ reparameterisation removes it.

4.4 What the change of variable buys

Arms A and B integrate the same ODE, so their ratio of errors measures the change of variable alone (Table 6). For RK4 the λ form is more accurate on every tier once the run reaches the data end, by factors of 48, 8551 and 10.6. Under the asymptotic protocol, which stops at $t = 0.2$, the factors are 0.47 and 1.77, which is why the analytic tiers first appeared indifferent to it. A grid uniform in t leaves one long final step across the region where $\sigma \rightarrow 0$, and RK4’s stages sample that region. For Euler and RK2 the effect is mixed: the λ form helps on Tier 2 by factors of 10 to 33 and is slightly worse on Tier 1 and on the network.

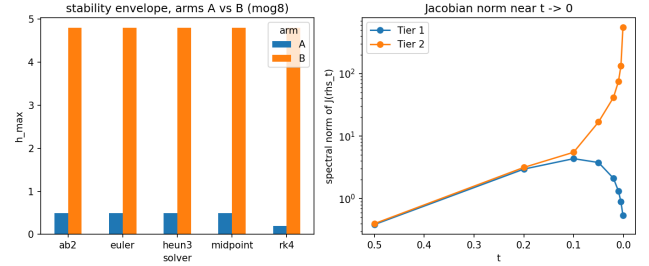


Figure 7: Stability on Tier 2. Left: largest stable step per stepper; bars at 0.4995 (in t) and 4.79 (in λ) are the coarsest grids tried, so those steppers never diverged, and only arm-A RK4 is limited. Right: spectral norm of the arm-A Jacobian along the trajectory; Tier 1 turns down near $t \rightarrow 0$ while Tier 2 climbs to 545.

Table 6: Error of arm A divided by error of arm B at the largest shared NFE. Values above 1 favour the λ grid.

Protocol	Tier	Euler	RK2	RK4
asymptotic	1	0.93	0.32	0.47
	2	0.92	0.39	1.77
practitioner	1	0.69	0.85	48.4
	2	9.92	33.1	8551
	3	0.63	0.56	10.6

4.5 When the exact linear treatment helps

The two analytic tiers disagree about DPM-Solver’s split. On Tier 1 ($\kappa = 10$), DPM-Solver-1 and -2 lose to the classical stepper of the same order at every budget from 10 to 120 NFE: at 20 NFE, DPM-Solver-2 errs by 0.133, RK2 in t by 0.034 and RK2 in λ by 0.0089. On Tier 2, at 12 NFE, DPM-Solver-2 errs by 0.084 against 0.23 (RK2 in λ) and 0.34 (RK2 in t).

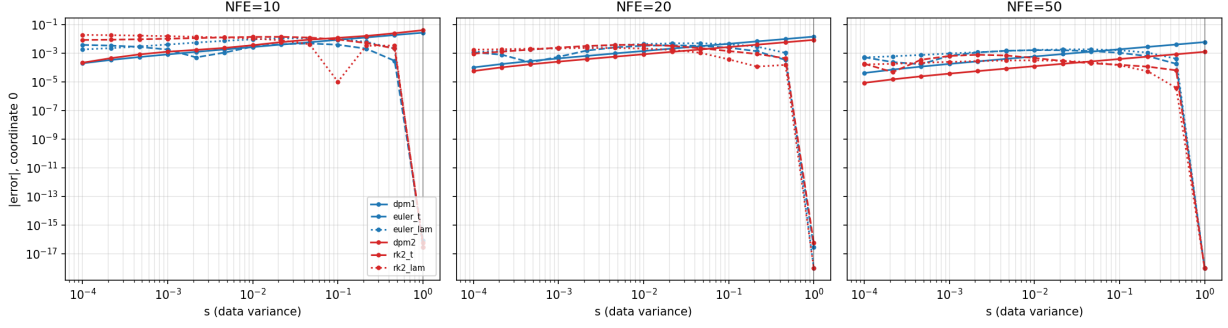


Figure 8: Error on one Gaussian coordinate of data variance s at 10, 20 and 50 NFE. Solid lines: DPM-Solver-1 and -2. Dashed and dotted: classical steppers of the same order in t and in λ . DPM-Solver leads for small s ; at $s = 1$ the classical right-hand side cancels and those steppers reach machine precision.

The difference is how concentrated the data is. For data concentrated at a point ($s \rightarrow 0$), $\varepsilon = x/\sigma$ is constant along the exact path, and substituting $e^h = \alpha_i \sigma_{i-1} / (\sigma_i \alpha_{i-1})$ into Eq. (6) gives $x_i = (\sigma_i / \sigma_{i-1}) x_{i-1}$, the exact solution. For data as spread out as the noise ($s = 1$), the right-hand side of Eq. (4) is identically zero and every classical stepper is exact, while $\varepsilon = \hat{\sigma}(\lambda)x$ still varies and DPM-Solver-1 is not. Sweeping s between the two (Figure 8), at 20 NFE and $s < 10^{-2}$ DPM-Solver-1 beats Euler in t at 83% of points and DPM-Solver-2 beats RK2 in t at all of them; at $s = 1$ the classical steppers are exact to 10^{-16} while DPM-Solver-1 errs by 5.9×10^{-3} . Natural images have a rapidly decaying pixel covariance spectrum, which puts most directions at the concentrated end and is consistent with DPM-Solver’s success on them. We have not measured that spectrum.

4.6 The crossover and Table 6

What is held fixed matters. Compared at the same step size, which gives DPM-Solver-3 three times the budget of DPM-Solver-1, the two curves cross at 4.4 to 5.0 NFE on Tier 1, flat across five decades of κ , and at 8.9 on Tier 2. The fair comparison, and the practical one, holds the budget fixed, so everything below is at matched NFE.

On this checkpoint the paper’s Table 6 gives DPM-Solver-3 an FID of 24.37 (at an actual 9 NFE) against 16.69 for DPM-Solver-1 at 10 NFE, and the order reverses by 12 NFE (8.20 against 13.63). With exact answers and a continuous budget we can locate the crossing (Table 7, Figure 9). On the real network, at matched NFE, DPM-Solver-3 overtakes DPM-Solver-1 at 14.1 NFE, close to the paper’s 10 to 12, with a trajectory-error metric instead of FID. The first crossing moves later as the problem gets harder: 5.1 on the linear Gaussian, 9.2 on the mixture and 14.1 on the network. This fits a pre-asymptotic explanation. At 9 NFE DPM-Solver-3 takes only three steps, and with steps that large its third-order correction increases the error. The larger the higher λ -derivatives of ε along the path, the smaller the step must be before the correction helps. Our own FIDs show the same reversal: DPM-Solver-3 scores 143 at 9 NFE against 44.9 for DPM-Solver-1 at 10, and 18.7 at 18 NFE against 27.0 at 20.

Table 7: NFE at which DPM-Solver-3 and DPM-Solver-1 cross, at matched NFE, practitioner protocol.

Testbed	Crossings	Note
Tier 1, Gaussian	5.1	one crossing
Tier 2, mog8	9.2, 14.2, 17.3	order 3 stays ahead from 17.3
Tier 3, CIFAR-10	14.1	paper’s checkpoint
Paper, Table 6 (FID)	10 to 12	same checkpoint

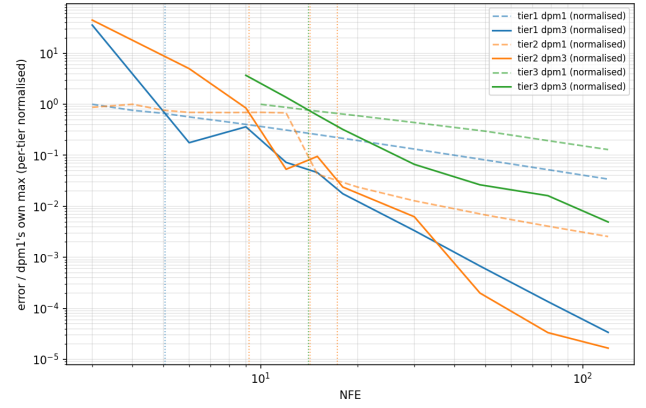


Figure 9: DPM-Solver-3 (solid) against DPM-Solver-1 (dashed) at matched NFE, each tier normalised by its own DPM-Solver-1 maximum. Dotted verticals mark crossings.

4.7 Tier-3 error and its components

Figure 10 shows Tier-3 error against NFE for all nine solver and arm combinations. Total error on Tier 3 has three parts, and each can be measured without the true score (Table 8, Figure 11). The floor, the disagreement between two references, is 0.368 in total, or 8×10^{-4} relative to $\|x\| \approx \sqrt{3072}$, which matches the expected float32 precision of about 10^{-3} . Stopping at $t_{\text{end}} = 2 \times 10^{-3}$ instead of 10^{-3} moves the result by 1.53, more than DPM-Solver-3’s whole discretisation error at 78 NFE (1.22). Beyond that budget, extra network calls reduce a component that the choice of t_{end} already

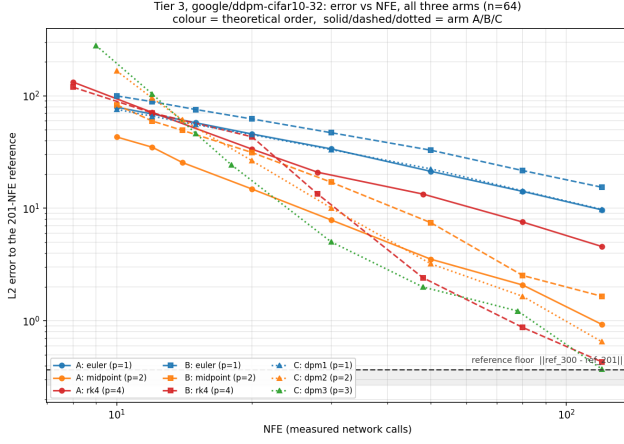


Figure 10: Tier 3: error against measured NFE for all nine solver and arm combinations. The dashed horizontal line is the reference floor $\|x_{300} - x_{201}\|$.

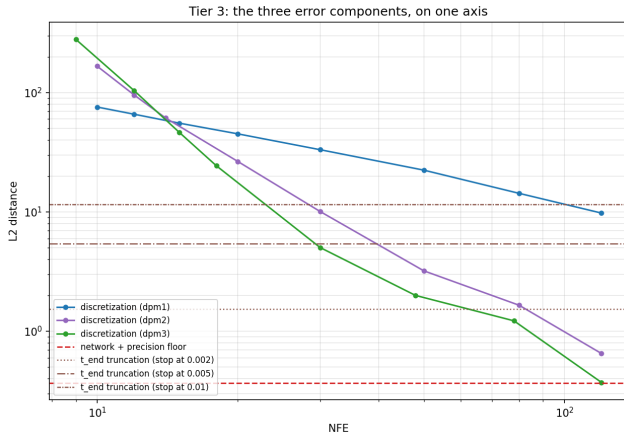


Figure 11: The three Tier-3 error components on one axis: discretisation error of DPM-Solver-1, -2 and -3 against NFE, the t_{end} truncation for stopping at 2, 5 and 10×10^{-3} (horizontal lines), and the reference floor (red, dashed). Once a discretisation curve falls below a truncation line, the choice of t_{end} matters more than further network calls.

outweighs. The paper’s Table 4 shows the same effect in FID: the type-2 DPM-Solver scores 4.39 at 15 NFE and 5.52 at 20.

4.8 Samples, FID and trajectory error

We rendered samples from six samplers at five budgets (Figure 12) and scored them with FID on 5,000 samples each, next to their L_2 distance to the converged image on a 64-image batch (Table 9).

Our FID ranking agrees with the paper’s. At about 10 NFE, order 2 beats order 1 (DPM-Solver-2 scores 24.9 against 44.9), order 3 loses to order 1 (143 against 44.9) and DPM-Solver-fast beats DDIM (31.3 against 39.5). Only the top pair is in a different order: DPM-Solver-2 edges DPM-Solver-fast here, while the paper has the reverse by

Table 8: The three components of Tier-3 error.

Component	Measured as	Value
discretisation	$\ x_N - x_{\text{ref}}\ $, DPM-Solver-3	279.6 at 9 NFE to 0.37 at 120
t_{end} truncation	$\ x_{\text{ref}}(t_{\text{end}}) - x_{\text{ref}}(10^{-3})\ $	1.53, 5.43, 11.56 at $t_{\text{end}} = 2, 5, 10 \times 10^{-3}$
network and precision floor	$\ x_{300} - x_{201}\ $	0.368

a small margin (6.42 against 7.90). This confirms that our Tier-3 setup reproduces the paper’s setting.

Trajectory error ranks the same samplers the other way. Among the five that produce an image at 10 NFE, ordering by L_2 (RK2 in t , DPM-Solver-1, DDIM, DPM-Solver-fast, DPM-Solver-2) is the exact reverse of ordering by FID, a rank correlation of -1.00 . DPM-Solver-2 lands 2.3 times further from the converged image than DPM-Solver-1 (180.6 against 78.4) and scores 1.8 times better FID. Figure 12 shows the reason. DPM-Solver-1 produces the right picture, blurred; DPM-Solver-2 produces a sharp picture of something different. A few large steps at high noise fix the layout early, and DPM-Solver-2 then denoises cleanly toward an image that is plausible but is not where this starting noise leads. FID rewards that outcome and L_2 penalises it. By 20 NFE the two metrics largely agree (rank correlation $+0.60$). The evidence therefore supports the paper’s claim that DPM-Solver produces good images in 10 NFE, while no sampler we ran integrates the ODE accurately at that budget: the closest, RK2 in t , is still 11 gray levels RMS from the converged image.

Absolute FID. Our FIDs are about three times the paper’s; even the converged run scores 16.5 against about 5.25. Part of this is the sample size. Real CIFAR-10 test images scored with the same FID call follow $\text{FID} \approx 3.0 \times 10^4 / N$ from 1,000 to 10,000 images (Figure 13), and at 10,000 they give 3.15, the value documented for clean-fid, which validates the pipeline. At our 5,000 samples the floor is 5.9; at the paper’s 50,000 it is about 0.6. Sample size therefore explains 5.2 of the 11.2-point gap. The remaining 6.0 points are a real difference that we did not isolate; candidates are clean-fid’s port of the Inception network and the conversion from discrete to continuous time. The floor shifts every sampler by the same amount, so it changes no ranking.

4.9 Summary against the base paper and our predictions

We wrote predictions down before running the experiments that test them: six before any sweep, and eight more after an internal review found that an early comparison had mixed the two protocols. The notebook notebooks/10_report.ipynb computes each verdict from the results files (Tables 10 and 11). Eleven were confirmed and three were narrowed. Stability turned out to depend on curvature and not on κ ; RK4 loses to DPM-Solver-3 per NFE only on Tier 3; and the FID sample-size floor explains about half of our gap to the paper’s absolute values.

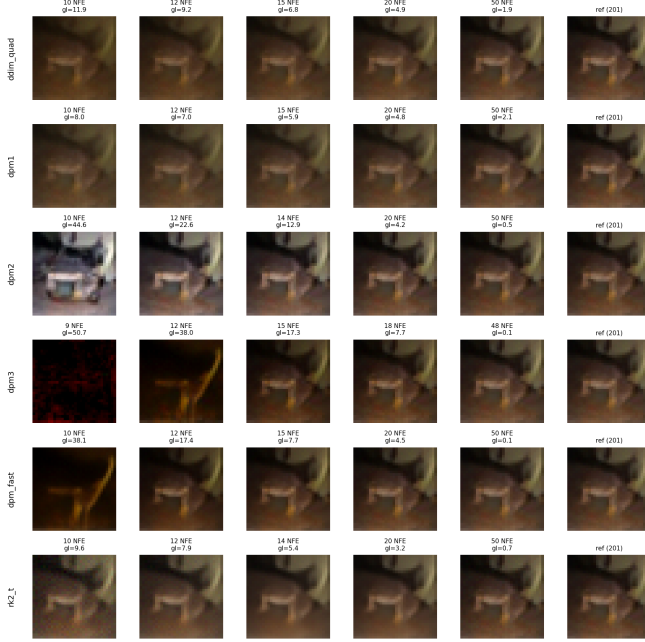


Figure 12: One starting noise, six samplers, five budgets, and the 201-NFE reference (right). “gl” is the mean absolute difference from the reference in 8-bit gray levels. At 10 NFE, DDIM and DPM-Solver-1 give the right picture, blurred; DPM-Solver-2 gives a sharp but different picture; DPM-Solver-3 at 9 NFE gives noise. By 20 NFE all samplers are visually close to the reference.

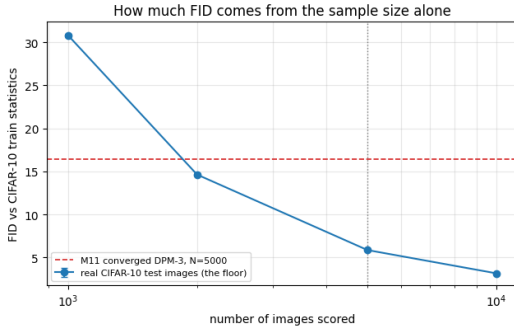


Figure 13: FID of real CIFAR-10 test images against sample size (three draws per size below 10,000, the whole test set at 10,000). The dashed line is our converged DPM-Solver-3 at 5,000 samples. Real images follow $\text{FID} \approx 3.0 \times 10^4 / N$, which sets a floor of 5.9 at our sample size.

5 Conclusion and future work

5.1 Conclusions

Measured as a numerical method, DPM-Solver behaves as its theory says wherever the theory applies. With analytic scores and small steps every solver we tested reaches its textbook order to within 0.1, and the published code is correct. The order theorem describes

Table 9: Image quality and trajectory accuracy on the same checkpoint. FID on 5,000 samples (ours) and the paper’s FID (Table 6; DDIM from Table 4; 50,000 samples). L_2 to the converged image over 64 images, and median per-image RMS in gray levels. DPM- k is DPM-Solver- k .

Sampler	NFE	FID	paper	L_2	gray
<i>about 10 NFE</i>					
DPM-2	10	24.9	7.90	180.6	49.3
DPM-fast	10	31.3	6.42	111.7	30.2
DDIM, quad.	10	39.5	13.58	91.9	25.8
DPM-1	10	44.9	16.69	78.4	21.9
RK2 in t	10	52.9	n/a	44.4	11.3
DPM-3	9	143.3	24.37	287.0	82.5
<i>about 20 NFE</i>					
DPM-2	20	15.6	5.24	27.3	7.1
RK2 in t	20	15.7	n/a	15.6	3.7
DPM-fast	20	18.0	5.52	18.7	4.1
DPM-3	18	18.7	5.43	26.4	6.4
DDIM, quad.	20	25.1	6.94	49.8	13.9
DPM-1	20	27.0	8.90	46.3	12.2
converged	99	16.5	~5.25		

Table 10: Predictions made before any sweep (C: confirmed, N: narrowed).

Prediction	Evidence
Tiers 1, 2 slopes match theory within ± 0.15	C worst 0.094, asymptotic protocol; up to 1.91 at practical budgets
Arm A degrades as κ grows, B and C do not	N no κ effect on the linear tier; RK4 limit on the curved tier
Euler in λ and DPM-1 both order 1, but differ	C slopes 1.02, 0.99; errors differ 1.4×
RK4 wins per step, loses per NFE to DPM-3	N only on Tier 3 (16 to 120 NFE); RK4 wins both on Tiers 1, 2
Tier-3 slopes fall below theory	C 8 of 9 more than 0.15 below; mostly the budget
Arm B can lose to arm A at low NFE	C at the coarsest shared budget, all tiers

the limit $h \rightarrow 0$, however, and at the budgets diffusion models are sampled at (10 to 120 NFE, down to $t_{\text{end}} = 10^{-3}$) no solver is in that regime: exact scores already miss theory by up to 1.9, against 2.8 on the network.

At those budgets two properties of the problem decide which sampler wins. The curvature of the data distribution destabilises explicit steppers in t and delays the point where order 3 overtakes order 1, from 5 NFE on a linear problem to 14 on the real network, which reproduces the paper’s Table 6 anomaly; the λ reparameterisation removes the instability and cuts RK4’s error by one to four orders of magnitude. The concentration of the data decides

Table 11: Predictions added after the internal review.

Prediction	Evidence
Exact-score tiers also miss theory at Tier 3's settings	C worst 1.91 against 2.79 on Tier 3
Curved tier: RK in t destabilises, λ form does not	C arm-A RK4 $h_{\max} = 0.1998$; no arm-B limit
λ form helps RK4 on every tier at $t_{\text{end}} = 10^{-3}$	C $48\times, 8551\times, 10.6\times$
Order 3 overtakes order 1 near 10 to 12 NFE	C 14.1 on the paper's checkpoint
Exact linear part helps concentrated, hurts spread-out data	C ahead at 83 to 100% of points for $s < 10^{-2}$; classical exact at $s = 1$
FID ranking at 10 NFE matches Table 6	C headline comparisons hold; top pair swapped
L_2 and image quality disagree at low NFE	C rankings exactly reversed at 10 NFE
Most of the FID gap is the 5k sample size	N about half (5.2 of 11.2 points)

whether DPM-Solver's exact linear treatment helps: it is exact for a point mass and a handicap for data as spread as the noise.

The metric matters as well. At 10 NFE, FID and distance to the converged image rank samplers in opposite orders. DPM-Solver's advantage at low NFE lies in producing a plausible image, and our FID ranking agrees with the paper's; at that budget no sampler integrates the ODE accurately. Applications that need the trajectory itself, such as reproducible sampling, interpolation or inversion, should budget 30 to 50 NFE: by 20 NFE the two rankings largely agree, and DPM-Solver-3 comes within one gray level of the converged image at about 36 NFE.

5.2 Limitations

All Tier-3 results come from one checkpoint at 32×32 resolution. Our FIDs use 5,000 samples, so we compare rankings and not absolute values, and half of the gap to the paper's absolute FIDs is unexplained. The FID samples were seeded through Python's per-process string hash, so they are not bit-reproducible and differences of a few tenths are noise. The Tier-3 reference is itself a numerical solution, so errors below its floor carry no information, and the practical Tier-3 sweep is pre-asymptotic by design. The concentration argument for natural images rests on their known covariance spectrum, which we did not measure.

5.3 Future work

Natural next steps are to repeat the Tier-3 study on larger and higher-resolution checkpoints, to compute FID on 50,000 samples with fixed seeds, to locate the remaining FID difference by comparing the paper's original discrete-time conversion with the interpolated schedule in the current code, to measure the pixel covariance spectrum of CIFAR-10 to test the concentration argument directly, and to extend the study to multistep solvers such as DPM-Solver++ and to adaptive step-size control.

Code availability. The public repository

<https://github.com/MehemudAzad/NUMERICAL-PROJECT>

contains the engine (src/), 129 unit tests, one notebook per experiment, every results file and figure, and run_all.sh, which

reproduces the laptop-side results from a clean clone. The Tier-3 notebooks run on a Kaggle T4.

References

- [1] John C. Butcher. 2016. *Numerical Methods for Ordinary Differential Equations* (3rd ed.). Wiley, Chichester, UK.
- [2] CSE 402 Section A, Group 03. 2026. A numerical analysis of DPM-Solver: code, notebooks and results. <https://github.com/MehemudAzad/NUMERICAL-PROJECT>.
- [3] John R. Dormand and Peter J. Prince. 1980. A Family of Embedded Runge–Kutta Formulae. *J. Comput. Appl. Math.* 6, 1 (1980), 19–26.
- [4] Stefan Elfving, Eiji Uchibe, and Kenji Doya. 2018. Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning. *Neural Networks* 107 (2018), 3–11.
- [5] Google. 2022. ddpm-cifar10-32: DDPM checkpoint trained on CIFAR-10. <https://huggingface.co/google/ddpm-cifar10-32>. Hugging Face model hub; port of the checkpoint released with Ho et al. (2020).
- [6] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. 1993. *Solving Ordinary Differential Equations I: Nonstiff Problems* (2nd ed.). Springer, Berlin.
- [7] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. Array Programming with NumPy. *Nature* 585 (2020), 357–362.
- [8] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. 2017. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30. Curran Associates, Inc., Red Hook, NY, USA, 6626–6637.
- [9] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 6840–6851.
- [10] Marlis Hochbruck and Alexander Ostermann. 2010. Exponential Integrators. *Acta Numerica* 19 (2010), 209–286.
- [11] John D. Hunter. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering* 9, 3 (2007), 90–95.
- [12] Kaggle. 2026. Kaggle Notebooks (free NVIDIA T4 GPU runtime). <https://www.kaggle.com/code>. Used for all Tier-3 runs.
- [13] Alex Krizhevsky. 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical Report. University of Toronto.
- [14] Cheng Lu et al. 2022. DPM-Solver: official PyTorch implementation. <https://github.com/LuChengTHU/dpm-solver>. File dpm_solver_pytorch.py, vendored unmodified at a pinned commit, MIT licence.
- [15] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. 2022. DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. Curran Associates, Inc., Red Hook, NY, USA, 5775–5787. arXiv:2206.00927.
- [16] Wes McKinney. 2010. Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference (SciPy)*. SciPy, Austin, TX, USA, 56–61.
- [17] Gaurav Parmar, Richard Zhang, and Jun-Yan Zhu. 2022. On Aliased Resizing and Surprising Subtleties in GAN Evaluation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, New Orleans, LA, USA, 11410–11420.
- [18] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32. Curran Associates, Inc., Red Hook, NY, USA, 8024–8035.
- [19] Jiaming Song, Chenlin Meng, and Stefano Ermon. 2021. Denoising Diffusion Implicit Models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Virtual.
- [20] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. 2021. Score-Based Generative Modeling through Stochastic Differential Equations. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Virtual.
- [21] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272.
- [22] Patrick von Platen, Suraj Patil, Anton Lozhkov, Pedro Cuenca, Nathan Lambert, Kashif Rasul, Mishig Davaadorj, and Thomas Wolf. 2022. Diffusers: State-of-the-Art Diffusion Models. <https://github.com/huggingface/diffusers>.

A Team contributions

- **Mehemud Azad** (2105014): orchestrated the team's work, put the members' parts together and made sure the whole pipeline runs end to end; planned all seven experiments (the implementation guide and the post-review correction plan); project setup and the shared results schema; the noise

schedule and the $\lambda \leftrightarrow t$ conversion; the Tier-1 Gaussian testbed; report writing: the rewrite of the technical report after the internal review, and the final revision of this report; the FID sample-size floor.

- **Sayjad Rahman** (2105021): the classical steppers and the integrator that measures NFE; the λ -uniform grids; DPM-Solver-1 and DDIM, the wrapper around the authors' code and the DDIM $\equiv$ DPM-Solver-1 check; the samples and FID on Kaggle; the control experiments (both protocols, Tier-2 stability, the matched-NFE crossover, when the exact linear part helps).
- **Gourove Roy** (2105017): Tier 3 on the real network, the shared engine that runs all three tiers, the report tables, the reproduction script and the reports.

Tier 3. The Tier-3 engine (`src/tier3.py`), which runs every solver on the CIFAR-10 DDPM network with the checkpoint's own discrete noise schedule and uses the identity $f = \sigma^2 \lambda'$ to keep arms A and B the same ODE (Section 2.4). **Notebook 09** (`notebooks/09_tier3_cifar10.ipynb`), run on a Kaggle T4: error against NFE for all nine solver and arm combinations, the cached reference runs, the reference's own error floor, and the split of Tier-3 error into discretisation, t_{end} truncation and that floor (Section 4.7, Figures 10 and 11, Table 8).

Shared engine. The integrator that runs on GPU tensors as well as NumPy arrays, and the probability-flow ODE written once with the schedule as a parameter, so the same code runs Tiers 1, 2 and 3.

Report tables. **Notebook 10** (`notebooks/10_report.ipynb`), which builds the master order table, the crossover summary, the predictions ledger and the proposal-coverage table directly from the results files (Tables 10 and 11, Figure 5); the floor-aware order fit that recovers DPM-Solver-3's order of 2.995 on the network (Section 4.2).

Reproducibility and reports. The script `run_all.sh`, run end to end, which reproduced the committed results to 1.8×10^{-15} ; the unit tests for Tier 3 and for the report tables; the first version of the technical report, and this ACM-format report.

- **Khalid Hasan Tuhin** (2105002): selected the project idea; designed and built the Tier-2 testbed (mog8, an 8-mode mixture on a Swiss roll whose exact score makes the probability-flow ODE nonlinear), with its DOP853 reference and convergence check, which every Tier-2 result in the order, stability and crossover sections runs on; the Tier-2 convergence experiment; the crossover study: the crossing finder that the matched-NFE crossover reuses, and the equal-step-size crossover on Tier 1 across five decades of κ and on Tier 2.
- **Niloy Das Robin** (2105019): the order-measurement and stability tools, and the Tier-1 studies built on them.

Order. The sliding-window order fitter, which finds the asymptotic part of a convergence curve on its own and reports the slope, fit window and R^2 ; the Tier-1 convergence experiment over all fourteen solvers in the three arms (Section 4.1, Figure 4).

Stability. The divergence test and the bisection for the largest stable step, validated against a linear ODE with a known stability threshold; the κ -sweep stability study, with a stress test to $\kappa = 10^8$ and $t_{\text{end}} = 10^{-6}$.

Analysis. The explanation of why the linear testbed never destabilises: the stability product $hJ(t_{\text{end}} + h)$ stays below about 0.9, under explicit Euler's threshold of 2 (Section 4.3).